

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ Информатика и системы управления

КАФЕДРА Программное обеспечение ЭВМ и информационные технологии

РУБЕЖНЫЙ КОНТРОЛЬ № 2

по предмету:
«Системы искусственного интеллекта»

Выполнил:

студент группы ИУ7-11М

Салженикин Д. А.

Подпись и дата:

Проверил:

преподаватель каф. ИУ7

Волкова Л. Л.

Подпись и дата:

Москва – 2025

Общий порядок выполнения лабораторной работы

1. Лабораторная работа №1. Формирование и предобработка датасета для рекомендательной системы
2. Лабораторная работа №2. Реализация метрик схожести
3. Лабораторная работа №3. Постановка лайков, дизлайков и ранжирование
4. Лабораторная работа №4. Иерархическая фильтрация по дереву и объединение с рекомендательной системой
5. Лабораторная работа №5. Составление корпуса запросов и сценариев
6. Лабораторная работа №6-8. Диалоговая система
7. Лабораторная работа №9: Лингвистическая переменная и Нечеткая логика

1. Лабораторная работа №1. Формирование и предобработка датасета для рекомендательной системы

Целью является создание структурированного, очищенного и готового к анализу набора данных, который станет основой для всех последующих алгоритмов рекомендаций и фильтрации.

Для представления данных был выбран формат CSV (файл test.csv) как наиболее универсальный и легко интегрируемый с основным инструментом анализа — библиотекой Pandas. Библиотека Pandas была выбрана в силу её эффективности для работы с табличными данными (DataFrame), мощных средств для фильтрации и манипуляций, что является критичным для быстрой работы фильтров и расчета метрик схожести.

Датасет был спроектирован с учетом требований гибридной рекомендательной системы:

Идентификационные признаки: ID, Название постановки (для уникальной привязки и взаимодействия).

- Категориальные признаки: Жанр, Театр, Стиль, Язык исполнения, Целевая аудитория (используются для иерархической фильтрации и расчета неевклидовой схожести).
- Числовые признаки: Рейтинг, Длительность (мин), Год постановки, Количество антрактов (используются для расчета расстояний и лингвистических переменных).
- Бинарные признаки: Платный?, Семейный?, Для детей? (используются как логические фильтры).
- Текстовые признаки: Теги, Темы (могут использоваться для будущих текстовых методов, но в данной системе — для контекста).

В файле `main.py` реализована функция `_load_data()`, которая выполняет следующие критические шаги:

Загрузка: `pd.read_csv('test.csv', decimal=',')`. Использование аргумента decimal=',' было необходимо, так как в русской локали принято использовать запятую в качестве десятичного разделителя (например, "7,5" вместо "7.5") для столбца Рейтинг. Без этой коррекции столбец был бы загружен как строковый, что сделало бы невозможным числовые сравнения.

Типизация. Было веявлено, что колонки, такие как Рейтинг и Длительность (мин), корректно преобразованы в числовой тип (float или int) для последующих математических операций.

Обработка пропусков: в реальных данных необходимо обрабатывать пропущенные значения (`NaN`). На этапе проектирования было принято решение обеспечить заполнение всех обязательных полей для стабильной работы алгоритмов расчета расстояний. Исходная база данных проиллюстрирована на рисунке 1.

ID	Название пос	Длительность	Год постанов	Количество а	Жанр	Язык исполне	Целевая ауд	Стиль	Проводится н	Семейный?	Для детей?	Платный?	Теги	Темы	Режиссер	Театр
1	Вишневый са	170	2015	1	Театр	Русский	Взрослая	Комедия	Нет	Нет	Нет	Да	классика, дра	утрата, перем	Владимир Ми	Театр ис
2	Не от мира се	100	2017	0	Семейный	Русский	Семейная	Классика	Нет	Да	Да	Да	семья, отнош	семейные цен	Екатерина По	Театр ис
3	Кармен	215	1999	2	Опера	Французский	Взрослая	Романтическа	Нет	Нет	Нет	Да	опера, любов	любовь, роко	Жорж Бизе	Музыка
4	Мадам Баттер	170	2002	1	Опера	Итальянский	Взрослая	Современная	Нет	Нет	Нет	Да	опера, трагед	любовь, пред	Джакомо Пуч	Музыка
5	Анна Болейн	240	2025	2	Опера	Итальянский	Взрослая	Классика	Нет	Нет	Нет	Да	опера, истори	власть, преда	Гаэтано Дони	Музыка
6	Красная Жизе	120	2025	1	Балет	Нет	Взрослая	Современный	Нет	Да	Да	Да	балет, танцы,	судьба, выбо	Жорж Бизе	Музыка
7	Не только лю	130	2024	1	Опера	Русский	Взрослая	Лирическая ко	Нет	Нет	Нет	Да	опера, любов	любовь, семе	Родион Щедр	Современ
8	Дуэль	150	2025	1	Театр	Русский	Взрослая	Психологичес	Нет	Да	Да	Да	драма, психо	столкновение	Лейла Абу-ал	Современ
9	Три товарища	180	1999	1	Театр	Русский	Взрослая	Классика	Нет	Нет	Нет	Да	классика, дру	дружба, войн	Галина Волче	Современ
10	Последняя ск	180	2025	1	Мьюзикл	Русский	Семейная	Сказка	Нет	Да	Да	Да	мюзикл, сказ	добро и зло, д	Дмитрий Бога	Московс
11	Вальс-бостон	180	2025	1	Мьюзикл	Русский	Взрослая	Драма	Нет	Нет	Нет	Да	мюзикл, драм	любовь, один	Михаил Мирс	Театр Э
12	Бременские м	100	2023	1	Театр	Русский	Детская	Музыкальный	Нет	Да	Да	Да	сказка, детск	приключения,	Евгений Марч	Театр ис
13	Женитьба Фи	180	2014	1	Театр	Русский	Семейная	Комедия	Нет	Да	Да	Да	классика, ком	любовь, интр	Евгений Писа	Театр ис
14	Атлантида. Т	120	2025	0	Иммерсивный	Русский	Семейная	Эксперимента	Нет	Да	Да	Да	иммерсивный	приключения,	Алексей Фра	Svet
15	Анна Каренин	150	2023	1	Мьюзикл	Русский	Взрослая	Классика	Нет	Нет	Нет	Да	мюзикл, класс	любовь, семь	Евгений Марч	Московс
16	Идиот	180	2017	1	Театр	Русский	Взрослая	Драма	Нет	Нет	Нет	Да	классика, дра	доброта, зло,	Юрий Еремин	Театр ис
17	Соло для час	130	2022	1	Театр	Русский	Взрослая	Трагикомедия	Нет	Нет	Нет	Да	трагикомедия	одиночество,	Павел Пархом	Театр ис
18	Севильский ц	170	2008	1	Опера	Итальянский	Взрослая	Комедия	Нет	Нет	Нет	Да	опера, класси	любовь, интр	Элайджа Мош	Театр Н
19	Двое на качел	180	2024	1	Театр	Русский	Взрослая	Драма	Нет	Нет	Нет	Да	драма, отнош	любовь, один	Галина Волче	Современ
20	Изумрудный г	85	2025	0	Мьюзикл	Русский	Детская	Шоу	Да	Да	Да	Нет	мюзикл, сказ	дружба, прик	Максим Рома	Планета
21	Мастер и Мар	200	2011	1	Театр	Русский	Взрослая	Классика	Нет	Нет	Нет	Да	классика, мис	добро и зло, л	Яношь Сас	МХТ им.
22	Не покидай се	110	2022	0	Театр	Русский	Взрослая	Фантазия	Нет	Нет	Нет	Да	фантастика, д	одиночество,	Виктор Краме	МХТ им.
23	Чайка	210	2024	1	Театр	Русский	Взрослая	Комедия	Нет	Нет	Нет	Да	классика, дра	творчество, л	Константин Х	МХТ им.
24	Сильфида	150	2021	1	Балет	Нет	Взрослая	Романтическ	Нет	Нет	Нет	Да	балет, роман	любовь, пред	Пьер Лакотт	Музыка
25	Евгений Онег	170	2007	1	Опера	Русский	Взрослая	Классическая	Нет	Нет	Нет	Да	опера, класси	любовь, судь	Александр Ти	Музыка
26	Норма	195	2023	1	Опера	Итальянский	Взрослая	Лирическая ко	Нет	Нет	Нет	Да	опера, класси	любовь, пред	Адольф Шапи	Музыка

Рис. 1. Исходная база данных

В результате ЛР1 мы получили DataFrame (`initial_df` в `recommendation_system.py`), который является чистой, структурированной и готовой к применению машинных алгоритмов базой данных.

2. Лабораторная работа №2. Реализация метрик схожести

Цель: оценка степени схожести театральных постановок на основе их количественных характеристик с использованием классических метрик расстояния.

2.1. Обоснование выбора метрики и подготовка данных

На начальном этапе проектирования рекомендательной системы был исследован потенциал метрического подхода, основанного на Евклидовом расстоянии. Данная метрика была выбрана как базовый, легко интерпретируемый метод для количественной оценки различий между объектами, представленными в многомерном числовом пространстве. Применение этого подхода требует отбора только тех признаков из исходного датасета (test.csv), которые являются непрерывными или дискретными числовыми величинами: Рейтинг, Длительность (мин), Год постановки и Количество антрактов.

Ключевым требованием для корректного применения Евклидова расстояния является нормализация выбранных числовых признаков. Без нормализации признаки с большим разбросом значений (например, Длительность от 60 до 240) оказывали бы непропорционально сильное влияние на итоговое расстояние, полностью нивелируя вклад признаков с узким диапазоном (например, Рейтинг от 7.0 до 9.5). В связи с этим, все выбранные числовые столбцы были приведены к безразмерному диапазону [0, 1] с помощью метода Min-Max масштабирования.

2.2. Применение Евклидова расстояния

Евклидово расстояние представляет собой геометрическое расстояние между двумя точками в N-мерном пространстве. После нормализации данных, расстояние между двумя спектаклями A и B (каждый из которых представлен вектором из N нормализованных числовых признаков) рассчитывалось по следующей формуле:

$$d(A,B) = \sqrt{\sum_{i=1}^{N} \left(x_{A,i} - x_{B,i}\right)^2} \quad (1)$$

где $x_{A,i}$ и $x_{B,i}$ — нормализованные значения i-го числового признака для спектаклей A и B соответственно. Чем ближе значение $d(A,B)$ к нулю, тем выше степень схожести объектов по их числовым параметрам.

2.3. Диагностика схожести с использованием Тепловой карты

Результатом попарного расчета Евклидова расстояния между всеми спектаклями в датасете является Матрица расстояний. Для визуального анализа этой матрицы и диагностики распределения схожести была построена Тепловая карта (Heatmap). Матрица расстояний представлена на рисунке 2.

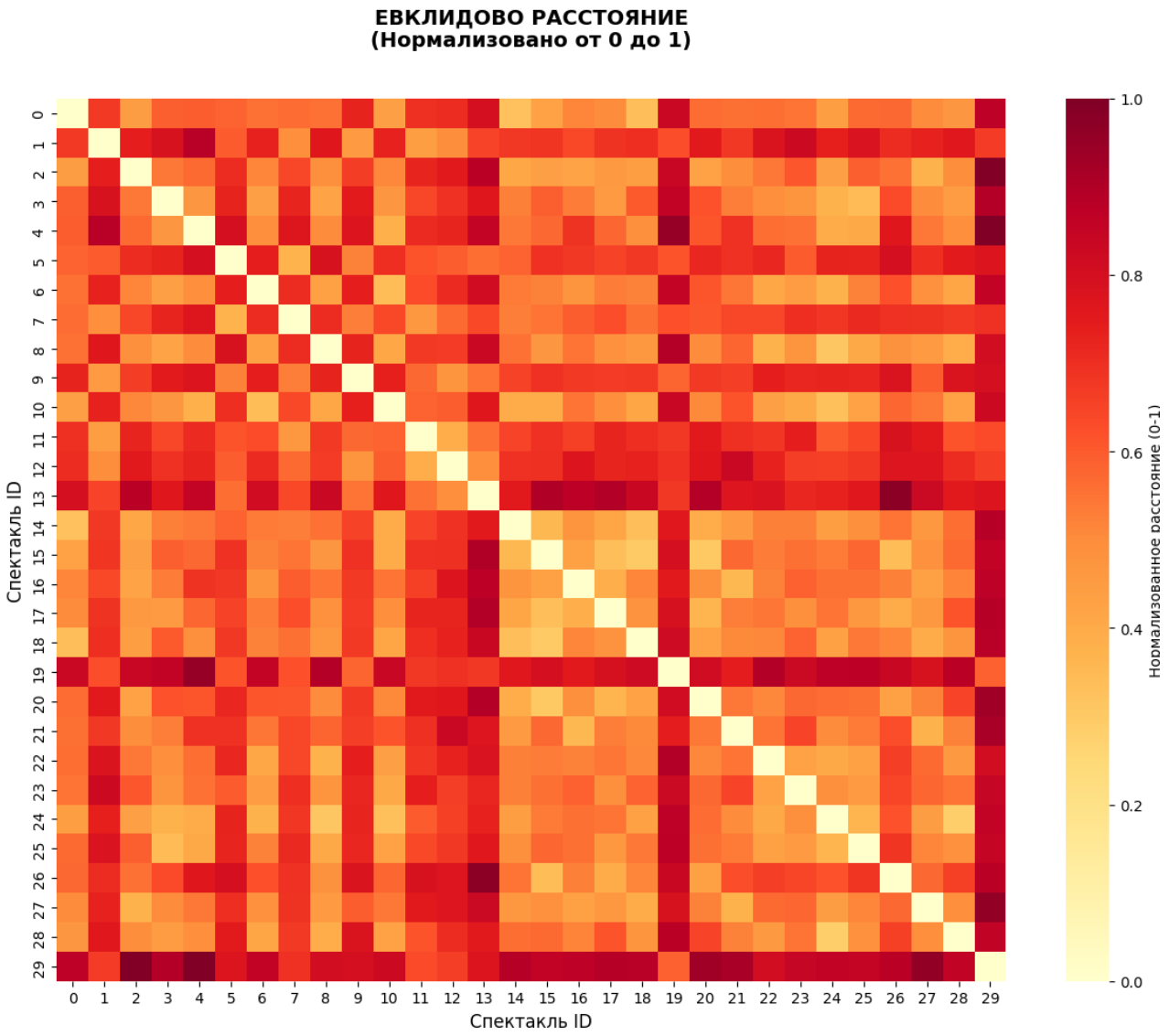


Рис. 2. Матрица расстояний

Тепловая карта представляет собой квадратную сетку, где каждая ячейка (i, j) окрашена в цвет, интенсивность которого прямо пропорциональна расстоянию $d(i, j)$:

- Холодные цвета (ближе к синему) указывают на малое расстояние и, следовательно, высокую схожесть спектаклей по их числовым параметрам (одинаковая длительность, схожий рейтинг и т.д.).
- Горячие цвета (ближе к красному/желтому) указывают на большое расстояние и существенное различие между объектами.

Анализ Тепловой карты позволил выявить естественные кластеры (группы спектаклей, образующие крупные "холодные" блоки на карте) и обнаружить выбросы (отдельные спектакли, показывающие высокую "горячую" удаленность от большинства других), которые, например, могут иметь экстремально большую длительность или необычно низкий рейтинг.

2.4. Выводы о применимости

Несмотря на эффективность в диагностике и простоту реализации, чистый Евклидов подход был признан недостаточным для полноценной рекомендательной системы в театральной области. Его ключевое ограничение заключается в невозможности обработки категориальных признаков (Жанр, Театр, Стиль), которые являются определяющими для пользователя. Спектакли с одинаковой длительностью, но разными жанрами ("Опера" и "Комедия") будут ошибочно признаны похожими, что приведет к некачественным рекомендациям.

2.5. Неевклидова метрика

Был реализован подход, основанный на Взвешенном многомерном расстоянии (неевклидова метрика), с использованием Матриц схожести для категориальных признаков.

В классе `InteractiveTreeRecommendationSystem` были определены:

A. Матрицы схожести (GENRE_SIMILARITY, STYLE_SIMILARITY и т.д.): это ручные словари (или матрицы), где экспертно задан коэффициент близости между парами значений. Например, ('Балет', 'Опера'): 0.8 означает высокую степень схожести, а ('Комедия', 'Опера'): 0.2 — низкую. Этот подход позволяет вводить предметное знание в математическую модель, делая рекомендации более осмысленными.

B. Веса признаков (FEATURE_WEIGHTS): для каждого столбца был задан вес, отражающий его важность в итоговом ранжировании (например, Жанр: 0.25, Рейтинг: 0.20, Длительность: 0.05). Этот механизм позволяет тонко настроить модель: пользователь скорее всего придаст большее значение Жанру, чем году постановки.

C. Единая формула схожести (`_calculate_similarity`): схожесть между двумя спектаклями $(S_{A,B})$ вычисляется как взвешенная сумма схожестей по всем N признакам:

$$S_{A,B} = \sum_{i=1}^{N} W_i \cdot S(A_i, B_i) \qquad (2)$$

Если признак категориальный, $S(A_i, B_i)$ берется из Матрицы схожести.Если признак числовой (Рейтинг, Длительность), он сначала нормализуется к диапазону [0, 1] относительно всего датасета, а затем схожесть вычисляется как $1 - \left|A_i^{\text{норм}} - B_i^{\text{норм}}\right|$, что является мерой близости.

Этот подход позволил создать математическую модель, которая учитывает как числовые различия, так и семантическую близость категорий, решая ключевую проблему работы с разнородными данными.

3. Лабораторная работа №3. Постановка лайков, дизлайков и ранжирование

Цель: создать ядро рекомендательной системы: построить профиль пользователя на основе его явных предпочтений (лайков и дизлайков) и использовать метрики схожести из ЛР2 для генерации рекомендаций.

В классе `InteractiveTreeRecommendationSystem` (далее — Система) реализовано управление состоянием пользователя:

- `self.likes`: Список ID спектаклей, которые понравились пользователю.
- `self.dislikes`: Список ID спектаклей, которые не понравились.

Профиль пользователя не хранится как явный вектор, а формируется динамически при каждом запросе на рекомендации: профиль пользователя P — это агрегированное представление (например, среднее) всех признаков лайкнутых спектаклей.

Метод `_update_ranking` выполняет основную работу по генерации рекомендаций:

1. Определение эталонного профиля: На основе ID из `self.likes` извлекаются соответствующие строки из DataFrame. Эти спектакли служат эталонами.
2. Вычисление схожести с эталонами: для каждого спектакля в текущем (возможно, отфильтрованном) DataFrame:
    - Система вычисляет его схожесть с каждым лайкнутым спектаклем, используя взвешенную формулу схожести из ЛР2.
    - Схожести усредняются, давая итоговый `Similarity_Score`.
3. Применение штрафов: Если спектакль совпадает по ключевым признакам с объектом из `self.dislikes`, применяется штрафной коэффициент (DISLIKE_PENALTY_FACTOR = 0.5). Это позволяет системе не исключать полностью целые категории, но снижать их приоритет, если они связаны с негативным опытом.
4. Финальное ранжирование: DataFrame сортируется по убыванию `Similarity_Score`, и первые N записей становятся рекомендациями (`self.ranking`).

Система также поддерживает интерактивные функции, необходимые для диалогового режима:

- Обработка ввода: Методы `process_like_dislike_and_rank` принимают ID (или название), добавляют его в соответствующий список и запускают обновление ранжирования.
- История и Откат (`undo_action`): В Системе используется двусторонняя очередь `self.action_history` (структура deque), в которой сохраняется копия ключевого

состояния системы (текущий DataFrame, лайки, дизлайки) перед каждым важным изменением. Это позволяет пользователю выполнить команду "Сделай шаг назад" или "Откати", восстанавливая предыдущее состояние.

3.1. Результаты запросов

На рисунке 3 и 4 представлены примеры сформированного ответа на вопросы.

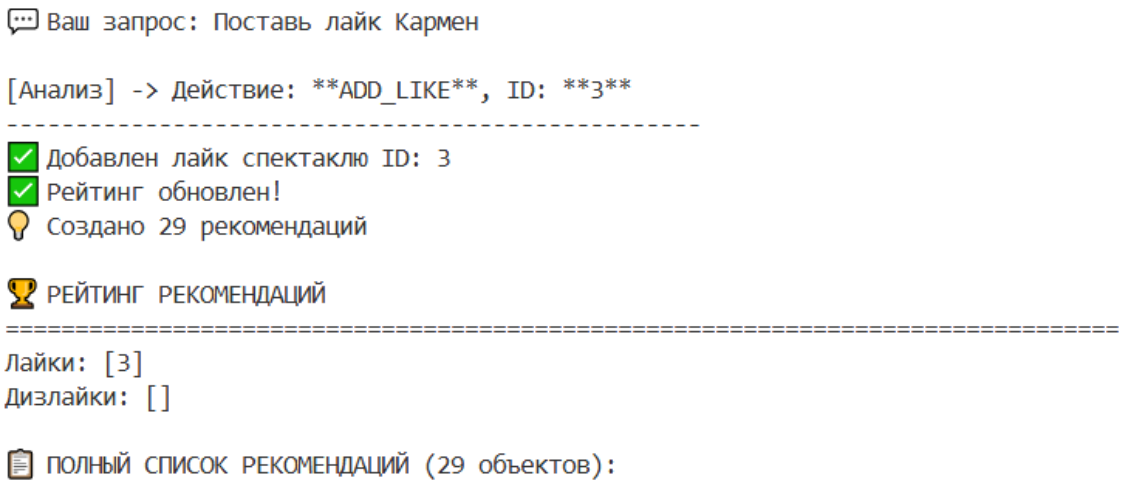


Рис. 3. Запрос на поставить лайк постановке

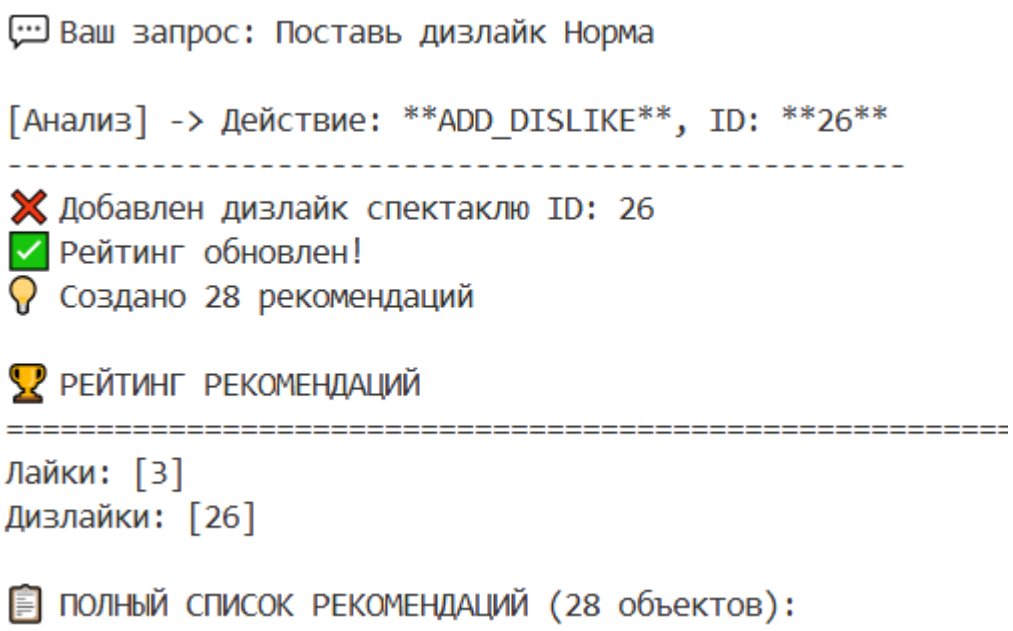


Рис. 4. Запрос на поставить дизлайк постановке

4. Лабораторная работа №4. Иерархическая фильтрация по дереву и объединение с рекомендательной системой

Цель: внедрить иерархический фильтр для сужения области поиска и объединить его с рекомендательным ядром ЛР3, создав гибридную систему.

Простой список фильтров (например, "Жанр: Опера И Театр: Большой") недостаточен. Если пользователь выбрал "Жанр: Балет", нет смысла предлагать ему "Театр: Малый театр", если там не идет ни одного балета. Это приводит к пустым результатам и фрустрации.

Выбранный метод: Дерево критериев (`CriteriaTree в criteria_tree.py`).

2. Реализация Дерева Критериев

- Класс `Node`: базовая структура, содержащая `value` (например, "Опера") и список `children` (следующий уровень иерархии).
- Класс CriteriaTree: инициализируется исходным DataFrame и упорядоченным списком категориальных колонок (`columns`).
- Рекурсивное построение (`_build_recursive`):
    1. Для первой колонки (Жанр) создаются узлы-дети для каждого уникального значения.
    2. Для каждого нового узла (например, узел "Опера") создается новый под-DataFrame, содержащий только оперы.
    3. Рекурсия продолжается для следующей колонки (Театр) на этом уменьшенном под-DataFrame.
    - Преимущество: если в под-DataFrame "Опера" нет ни одного спектакля из "Театра X", то узел "Театр X" просто не будет создан в ветке "Опера". Это обеспечивает автоматическую каскадную фильтрацию возможных критериев.

3. Гибридная система (ЛР3 + ЛР4)

В `recommendation_system.py` реализован метод `_apply_filter` (или `_apply_natural_filter`), который выполняет гибридную логику:

1. Жесткая фильтрация: На основе запроса, преобразованного в фильтры (см. ЛР6), система создает маску для исходного DataFrame и формирует новый, уменьшенный DataFrame (`self.current_df`).
2. Мягкое ранжирование: только после этого, на этом уменьшенном наборе данных, вызывается функция `_update_ranking` из ЛР3.

Система сначала отсекает все, что не соответствует жестким критериям фильтра (например, "только оперы"), а затем внутри этой подвыборки рекомендует самые похожие спектакли на основе лайков. Это сочетает точность фильтра с персонализацией рекомендаций.

4.1. Результаты запросов

На рисунке 5 представлен пример сформированного ответа на вопрос.

```
💬 Ваш запрос: Найди спектакли где Жанр Опера и Рейтинг выше 8.0

[Анализ] -> Действие: **ADVANCED_SEARCH**, ID: **None**
------------------------------------------------
🌳 ФИЛЬТРАЦИЯ ПРИМЕНЕНА (с использованием CriteriaTree)
✅ Фильтр по **Жанр**: == Опера
✅ Фильтр по **Рейтинг**: > 8.0
💡 Найдено 3 записей.

🎭 ОТФИЛЬТРОВАННЫЕ СПЕКТАКЛИ:
+----+------+-----------------------+--------+-----------+----------------------+---------------------
|    |   ID | Название постановки   | Жанр   |   Рейтинг |   Длительность (мин) | Театр
+====+======+=======================+========+===========+======================+=====================
|  3 |    4 | Мадам Баттерфляй      | Опера  |       8.9 |                  170 | Музыкальный театр им
+----+------+-----------------------+--------+-----------+----------------------+---------------------
| 17 |   18 | Севильский цирюльник  | Опера  |       9.1 |                  170 | Театр Новая Опера
+----+------+-----------------------+--------+-----------+----------------------+---------------------
| 25 |   26 | Норма                 | Опера  |       9.2 |                  195 | Музыкальный театр им
+----+------+-----------------------+--------+-----------+----------------------+---------------------
✅ Рейтинг обновлен!
💡 Создано 2 рекомендаций

🏆 РЕЙТИНГ РЕКОМЕНДАЦИЙ
================================================================================
Лайки: []
Дизлайки: [26]
Активные фильтры: Жанр: == Опера, Рейтинг: > 8.0
```

Рис. 5. Запрос на “Найди спектакли где Жанр Опера и Рейтинг выше 8.0”

5. Лабораторная работа №5. Составление корпуса запросов и сценариев

Цель: собрать репрезентативный набор пользовательских фраз для определения функциональных требований и тестирования модуля обработки естественного языка (NLP).

Были определены основные интенты пользователя, характерные для рекомендательной системы:

- Filter/Search: "Покажи комедии с рейтингом выше 8.0".
- Like/Dislike: "Поставь лайк спектаклю Кармен".
- Control/Maintenance: "Сделай шаг назад", "Сбрось фильтры".
- Question/Status: "Покажи мои лайки", "Сколько еще спектаклей осталось?".

Далее буду продемонстрированы сформированные возможные вопросы к системе:

1. Рекомендательная система

- Какие **пять лучших постановок** вы мне порекомендуете?
- Поставь(те) дизлайк на ХХХ, лайк на УУУ и лайк на ЧЧЧ. Что порекомендуете?
- Покажи топ-5 рекомендаций
- Я лайкнул постановку Кармен. Дай мне три рекомендации, которые наиболее похожи на неё по Целевой аудитории и Языку исполнения.

2. Поиск

- Какие постановки с Рейтингом выше 8.0 идут в Музыкальном театре им. К.С. Станиславского?
- Найди все спектакли в жанре Театр и Комедия поставленные до 2000 года
- Покажи мне все записи, где Язык исполнения — Русский а Целевая аудитория — Семейная
- Существуют ли постановки с Количеством антрактов 0 которые не являются Платными?
- Покажи все оперы и мюзиклы

3. Сравнение

- Какой театр имеет самый высокий Рейтинг среди всех постановок в базе данных?

4. Общие вопросы (это как панель из help, как общие вопосы)

- Как мне начать пользоваться вашей системой рекомендаций?
- Объясните, я могу ставить лайки и дизайки всем постановкам.
- Каковы основные правила вашей системы (например, как работают лайки, дизлайки и сброс настроек)?
- Что произойдет, если я попытаюсь лайкнуть уже понравившийся мне спектакль?
- Какой самый высокий вес (важность) присвоен любому признаку в расчете схожести?

5. Шаг назад по истории вопросов

- Пожалуйста, используйте команду "шаг назад", или явно снимайте фильтры, лайки и/или дизлайки.
- Сделай шаг назад
- Сбрось все ...
- Откати на шаг назад
- Сними фильтр ...
- Сними (диз)лайк ...

6. Иное, на что дается шаблонный ответ (На такие вопросы дается ответ: "Прошу прощения, я не могу ответить на ваш вопрос.")

- Назови режиссера, который в 1987 году поставил Палату №6 в Лондоне.
- Можешь ли ты спрогнозировать кассовые сборы этих постановок на следующий год?
- Дай мне список всех постановок, отсортированных по цвету их декораций.
- Сможешь ли ты загрузить в систему новые данные из внешнего источника, чтобы обновить Теги

- Как мне оформить возврат билета на спектакль Кармен?

2. Роль в разработке NLP

Этот этап, будучи не кодированием, является фундаментальным для ЛР6:

Обучение `ActionTagger`: корпус определил, какие глаголы и ключевые слова (например, "покажи", "найди", "выведи", "поставь", "удали") должны быть лемматизированы и сопоставлены с конкретными действиями (`ACTION_MAP` в `action_tagger.py`).

Тестирование `EntityExtractor`: определил необходимость обработки опечаток в названиях ("Крмен" вместо "Кармен") и множественных форматов ID ("ID 5", "номер 5", "спектакль 5").

Построение грамматики `QueryToFilterConverter`: Помог выявить сложные конструкции с союзами ("и", "или") и сравнительными оборотами ("выше 8", "менее 90 минут"), которые должны быть преобразованы в логические операторы (&, |, >, <).

Корпус запросов стал основным набором юнит-тестов для проверки робастности и полноты NLP-модулей.

6. Лабораторная работа №6-8. Диалоговая система

Цель: разработать многоуровневый NLP-конвейер для преобразования неструктурированного русского текста в структурированные команды, которые может выполнить ядро системы.

6.1. Лексический и морфологический анализ (Библиотека PyMorphy2)

Для работы с русским языком была выбрана библиотека PyMorphy2.

- Почему PyMorphy2: русский язык является высокоинфлективным (богат на падежи, числа, времена). Фраза "Покажи оперу" содержит слово "оперу" (винительный падеж), а в базе данных оно записано как "Опера" (именительный падеж). Прямое сопоставление невозможно.
- Реализация (`MorphologicalAnalyzer в nlp_analyzers.py`):
    - Основной метод — `normalize(word)` — выполняет лемматизацию: приводит слово к его нормальной (словарной) форме.
    - Пример: `normalize`("спектакли") → "спектакль". Это позволяет `ActionTagger` и `EntityExtractor` искать ключевые слова по их леммам, игнорируя грамматические формы.

6.2. Синтаксический анализ и извлечение фраз (Библиотека SpaCy)

Для понимания структуры предложения и извлечения смысловых групп была выбрана библиотека spaCy с русской моделью (`ru_core_news_sm`).

- Почему SpaCy: она позволяет строить дерево синтаксических зависимостей и определять части речи.
- Реализация (`SyntacticAnalyzer в nlp_analyzers.py`): анализатор принимает запрос и извлекает именные группы (noun chunks), которые часто содержат целевые критерии фильтрации (например, "длинные спектакли", "лучшая комедия").

6.3. Распознавание интентов (`ActionTagger`) и сущностей (`EntityExtractor`)

Эти модули служат мостом между лингвистическим анализом и командами системы:

- `ActionTagger`: использует лемматизированный запрос и `ACTION_MAP` (ручной словарь ключевых слов) для однозначного определения интента (намерения). Если не находит прямого совпадения, он использует контекстный анализ (например, наличие слов "лайк" и "удалить" одновременно → `REMOVE_LIKE`).
- `EntityExtractor`:
  - Извлечение ID: использует регулярные выражения для поиска числовых ID.
  - Нечеткий поиск названий (Fuzzy Matching): для обработки опечаток (ключевое требование диалоговой системы) используется библиотека Rapidfuzz (`fuzzy_matching.py`). Метод `find_id_by_title` принимает неточное название ("Крмен") и сравнивает его со всеми названиями в базе данных, используя расстояние Левенштейна для нахождения наиболее похожего совпадения с порогом схожести (например, 70/100).

6.4. Парсинг Запроса в Фильтр (`QueryToFilterConverter`)

Самый сложный модуль, отвечающий за преобразование текста в исполняемый код для Pandas:

- Парсер ищет ключевые слова, связанные со столбцами (`COLUMN_MAP`), операторами сравнения (леммы слов "выше", "меньше") и значениями.
- Он обрабатывает логические связки ("и" → &, "или" → |).
- Результат: преобразование фразы "комедии с рейтингом выше 8.0" в список структур: `{'column': 'Жанр', 'operator': '==', 'value': 'Комедия'}` И `{'column': 'Рейтинг', 'operator': '>', 'value': 8.0}`. Эти структуры затем используются в recommendation_system.py для фильтрации DataFrame.

На рисунке 6 представлен пример начального состояния системы.

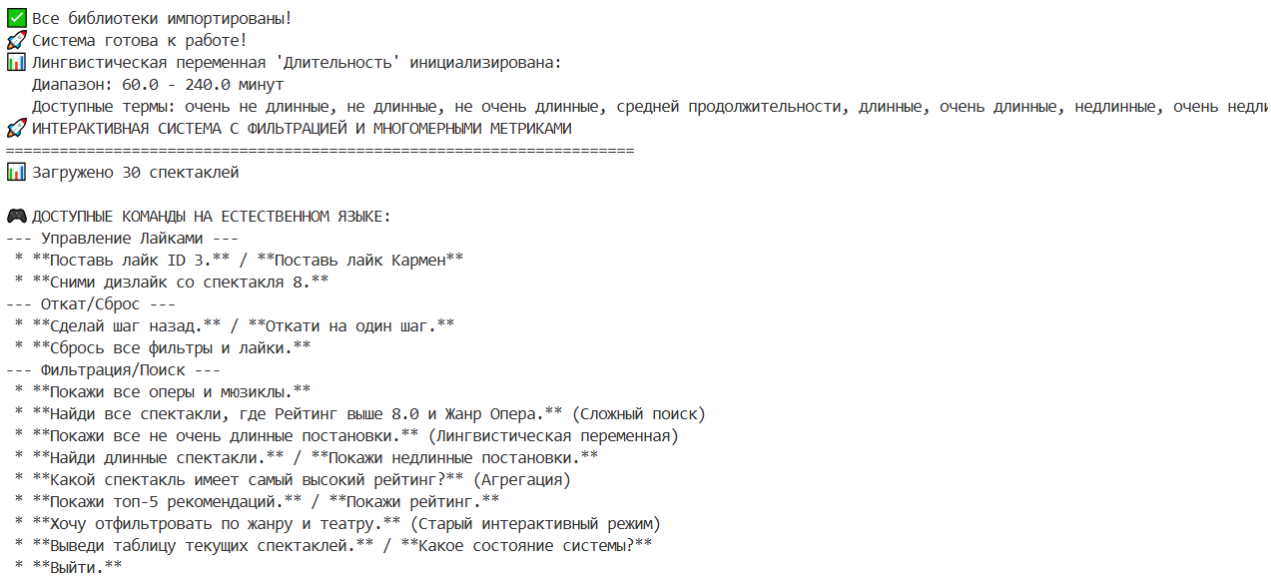


Рис. 6. Начальное состояние диалоговой системы

На рисунке 7 и 8 представлены примеры запросов к системе.

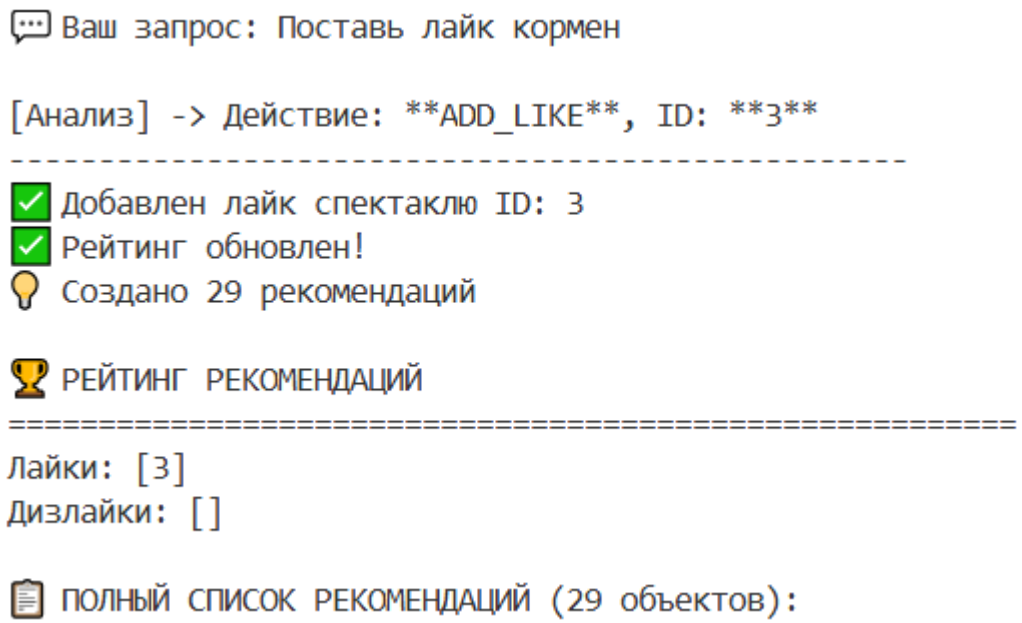


Рис. 7. Запрос для демонстрации обработки ошибок в запросах

```
💬 Ваш запрос: Как начать пользоваться системой?

[Анализ] -> Действие: **GENERAL_QUESTION**, ID: **None**
--------------------------------------------------

📖 КАК НАЧАТЬ ПОЛЬЗОВАТЬСЯ СИСТЕМОЙ:
1. Добавьте лайки к понравившимся спектаклям (например: 'Поставь лайк Кармен')
2. Добавьте дизлайки к непонравившимся спектаклям
3. Система автоматически создаст рейтинг рекомендаций на основе ваших предпочтений
4. Используйте фильтры для поиска спектаклей по критериям
5. Используйте 'шаг назад' для отката последнего действия
```

Рис. 8. Запрос на возможности диалоговой системы

7. Лабораторная работа №9. Лингвистическая переменная и Нечеткая логика

Цель: внедрить элементы нечеткой логики для обработки неточных, субъективных запросов пользователя, связанных с числовыми характеристиками (например, "длинный", "не очень высокий").

1. Суть Лингвистической Переменной

Традиционные фильтры требуют четких границ (например, Длительность > 180). Однако в реальном диалоге пользователь скажет: "Покажи длинные спектакли". Для системы рекомендаций, имитирующей человека, необходимо понимать эти лингвистические термы.

Выбранный метод: класс `LinguisticVariable` по методологии Нечетких множеств.

2. Реализация Лингвистической Переменной

- Термы и Функции Принадлежности: для числовой переменной (Длительность (мин)) были определены лингвистические термы: "очень короткий", "короткий", "средний", "длинный", "очень длинный".
- Определение Диапазонов (на основе респондентов): Числовые границы для этих термов были определены не произвольно, а на основе эмпирического подхода, имитирующего опрос респондентов (использование массивов `x_points` и `mu_base` в методе `_define_terms`). Эти массивы задают точки, где функция принадлежности $\mu(x)$ достигает пика или пересекается с соседними термами.
- Нечеткие модификаторы: Реализована обработка нечетких модификаторов ("очень", "не очень"). Например, "очень длинный" соответствует более строгому (суженному) диапазону в пределах терма "длинный".
- Конвертация в Фильтр: Метод `get_filter_from_term` преобразует лингвистический терм ("длинный") в четкий числовой диапазон (например, [150, 240])

и возвращает фильтр Pandas: `df['Длительность (мин)'] >= 150 & df['Длительность (мин)'] <= 240`.

Внедрение лингвистических переменных позволило Системе понимать нечеткие запросы, повысив её естественность и удобство для конечного пользователя.

7.1. Результаты анкетирования

Опрос проводился среди студентов группы ИУ7-11М. В таблице 1 приведены результаты анкетирования. 2 — сводная таблица результатов анкетирования.

Таблица 1 — Результаты анкетирования

Эксперт	Терм	Длительность постановки (мин)											
		60	85	100	120	130	150	170	180	195	200	215	240
1	Очень недлинные	1	1										
	не длинные			1									
	не очень длинные				1								
	средней продолжительности					1	1						
	длинные							1	1				
	очень длинные									1	1	1	1
2	Очень недлинные	1	1										
	не длинные			1									
	не очень длинные				1	1							
	средней продолжительности						1	1					
	длинные								1	1			
	очень длинные										1	1	1
3	Очень недлинные	1	1										
	не длинные			1	1								
	не очень длинные					1							
	средней продолжительности						1	1					
	длинные								1	1			
	очень длинные										1	1	1
4	Очень недлинные	1	1	1									
	не длинные				1								
	не очень длинные					1							
	средней продолжительности						1	1	1	1			
	длинные										1	1	
	очень длинные												1

Таблица 2 — Сводная таблица результатов анкетирования

Терм	Длительность постановки (мин)											
	60	85	100	120	130	150	170	180	195	200	215	240
Очень недлинные	1	1	1/4									
не длинные			3/4	1/2								
не очень длинные				1/2	3/4							
средней продолжительности					1/4	1	3/4	1/4	1/4			
длинные							1/4	3/4	1/2	1/4	1/4	
очень длинные									1/4	3/4	3/4	1

Путем нахождения точек пересечения графиков были получены следующие минимальные и максимальные значения для длительности постановки для каждого терма:

— Очень недлинные, до 95 мин

— Не длинные, от 95 мин до 120 мин

— Не очень длинные, от 120 мин до 136.67 мин

— Средней продолжительности, от 136.67 мин до 176.67 мин

— Длинные, от 176.67 мин до 195 мин

— Очень длинные, с 195 мин и далее

7.2. Примеры сформированных ответов на вопросы

На рисунке 9 представлен пример сформированного ответа на вопрос.

```
💬 Ваш запрос: Выведи все очень не длинные спектакли

[Анализ] -> Действие: **ADVANCED_SEARCH**, ID: **None**
--------------------------------------------------
🌳 ФИЛЬТРАЦИЯ ПРИМЕНЕНА (с использованием CriteriaTree)
✅ Фильтр по **Длительность (мин)**: >= 60 и <= 95.0
💡 Найдено 2 записей.

🎭 ОТФИЛЬТРОВАННЫЕ СПЕКТАКЛИ:
+----+------+-------------------------------+---------+-----------+---------------------+-----------------------+
|    |   ID | Название постановки           | Жанр    |   Рейтинг |   Длительность (мин) | Театр                 |
+====+======+===============================+=========+===========+=====================+=======================+
| 19 |   20 | Изумрудный город              | Мьюзикл |       8.9 |                  85 | Планета КВН           |
+----+------+-------------------------------+---------+-----------+---------------------+-----------------------+
| 29 |   30 | Учёная семейка и квадрокоптер | Театр   |       9.6 |                  60 | Центральный дом актера |
+----+------+-------------------------------+---------+-----------+---------------------+-----------------------+

📊 ТЕКУЩЕЕ СОСТОЯНИЕ СИСТЕМЫ
==========================================
- Исходное количество спектаклей: 30
- Текущее количество спектаклей: 2
- Лайки: []
- Дизлайки: []
- Активные фильтры: {'Длительность (мин)': '>= 60 и <= 95.0'}
- Есть рейтинг: Нет
```

Рис. 9. Запрос на вывод очень недлинных спектаклей

Выводы

Разработанная система рекомендаций является многомодульным решением, объединяющим методы машинного обучения, лингвистического анализа и нечеткой логики.

В основе системы лежит ЛР1, где была проведена критическая предобработка данных из test.csv, включая корректную загрузку числовых полей с десятичными запятыми, что заложило основу для всех последующих вычислений.

Ключевым интеллектуальным достижением стала ЛР2: вместо ограниченного Евклидова расстояния была реализована взвешенная неевклидова метрика схожести. Эта метрика позволяет системе учитывать семантику категориальных признаков с помощью экспертно заданных Матриц Схожести, объединяя их с нормализованными числовыми различиями для получения общего, более релевантного показателя близости постановок.

На базе этой метрики в ЛР3 был реализован алгоритм Content-Based ранжирования. Система динамически формирует профиль пользователя на основе его явных лайков и дизлайков, используя веса и штрафы для создания, персонализированного топ-листа рекомендаций. Функционал отката на основе очереди обеспечил интерактивность и удобство управления предпочтениями.

Гибридность системы была дополнена в ЛР4 внедрением Дерева Критериев, которое обеспечивает иерархическую фильтрацию. Это позволяет системе сначала жестко сужать выборку по заданным категориям, а затем применять мягкое ранжирование по схожести только к оставшемуся подмножеству, совмещая точность поиска с персонализацией.

Последующие работы (ЛР5 и ЛР6) сфокусировались на интерфейсе, основанном на естественном языке. Был разработан полный NLP-конвейер: Морфологический анализатор (PyMorphy2) обеспечивает лемматизацию для русского языка, ActionTagger определяет интент пользователя, а EntityExtractor использует нечеткий поиск для извлечения названий спектаклей с учетом опечаток. QueryToFilterConverter преобразует сложные текстовые запросы в логические выражения для фильтрации данных Pandas.

Завершающая ЛР7 добавила системе гибкости через Нечеткую логику. Была реализована концепция Лингвистической Переменной, что позволяет пользователю использовать субъективные термы. Система интерпретирует эти термы, используя функции принадлежности, и преобразует их в четкие числовые диапазоны для фильтрации, делая диалог с системой более естественным.

В результате был создан интегрированный комплекс, который эффективно обрабатывает неструктурированные запросы и предоставляет высокорелевантные рекомендации, основанные на гибридном методе ранжирования и фильтрации